上海海事大学

本科生
课程报告

课程名称：Pytorch 深度学习框架

设计题目：AOT-GAN 图像修复

学　　院：信息工程学院

姓　　名：钱毅　　学 号：202110310148

专　　业：人工智能　　班 级：211

成　　绩：

目　录

一、项目概述 1
1.1 流程图与总体方法 1
1.2 生成对抗网络简介 2
二、AOT-GAN 网络的构建 3
2.1 AOT 模块构建 3
2.2 整体网络结构 5
2.3 生成器网络结构 6
2.4 判别器网络结构 7
2.5 软掩码引导 PatchGAN(SM-PatchGAN) 8
2.6 相关激活函数 9
三、损失函数 10
四、数据集及其预处理 11
4.1 人脸数据集介绍 11
4.2 数据预处理 11
4.2.1 数据选择 13
4.2.2 数据操作 13
五、实现细节 14
5.1 实验环境配置 14
5.1.1 硬件环境 14
5.1.2 软件环境 14
5.2 训练过程 14
六、实验结果分析 14
6.1 损失函数结果 14
6.2 直观修复感受 15
6.3 部分修复结果展示 15
6.4 修复案例分析 16
七、总结与展望 17
参考文献 18

摘　要:本文利用改进版GAN生成对抗式网络——聚合上下文变换GAN(AOT-GAN)来实现对于高分辨率图像(512×512)的修复任务。目前，基于生成对抗式网络的方法在图像修复任务上取得了较好的效果，但在高分辨率图像中可能会产生扭曲的结构和模糊的纹理。为了增强上下文推理，通过将提出的AOT块堆叠多层来构建AOT-GAN生成器。AOT阻止来自来自不同接受域的聚合上下文转换，允许捕获信息丰富的远距离图像上下文和丰富的上下文推理模式。再通过定制的掩膜预测任务训练AOT-GAN的鉴别器来增强它。这样的训练目标迫使鉴别器区分真实和合成补丁的细节外观，进而促进生成器合成清晰的纹理。在CelebMask数据集的人像修复任务上起到了很好的效果。

关键词:生成式对抗网络(GAN);图像修复;计算机视觉;深度学习

一、项目概述

1.1 流程图与总体方法

图 1-1 总流程图

如图1-1所示,本文首先构建特定的AOT模块，用于聚合多个上下文转换以增强上下文推理，此外，AOT块的堆栈再很大程度上丰富了生成器网络中不同路径的组合，允许生成器

捕捉尽可能多的上下文感兴趣推理模式。其次，基于构建的 AOT 块构建生成对抗式网络模型，其中，生成器通过堆叠多层精心设计的 AOT 来构建，判别器由量身定制的掩码预测任务改进，该任务可反过来促进生成器合成清晰的纹理。再者，根据网络结构，对数据集进行预处理，数据集包括 30000 张高分辨率人脸图像，每张人脸包括若干面部掩码图，共计约 300000 张，在数据与处理中，将图片对应的掩码按特定方式拼接结合、增强、标准化作为模型的输入。接着对生成器和判别器构建特定损失函数以进行训练。最后，在测试中使用带掩码的人脸图像进行测试，评判模型复原图像的效果。

1.2 生成对抗网络简介

生成对抗网络的原理[10]是两个网络互相以相反的目的进行学习优化自身，即将两网络形成竞争关系。

这两个网络一个是输入为待修复图片，输出为修复图片的生成器网络 G。

另一个网络是输入为一副图片，输出为可能是原图片的概率的判别网络 D。如果确定是原图片的会输出会接近 1，如果确定是生成图片则输出会接近 0.

理想情况下最终到达判断器无法确定是假图片还是原图的平衡。

很容易想到的问题就是，为什么目标不是让判断器判别成输入图片是原图呢？因为如果判断器认为生成的假图片高概率为原图。必然证明了判断器自身精确度不够，被认为是原图的假图片效果也不一定好，判断结果就不可信。图 2-2 是 GAN 网络的对抗生成原理图。

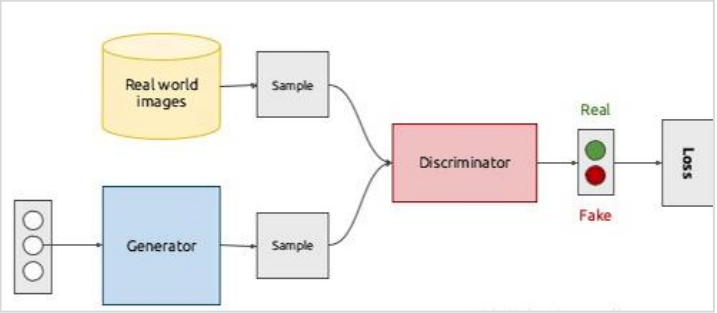


图 2-2　GAN 网络生成图片原理图　（输入随机噪声使生成器生成的假图片和真图片又作为判断器的输入，判断器判别错误惩罚判断器，判别正确惩罚生成器）

二、AOT-GAN 网络的构建

2.1 AOT 模块构建

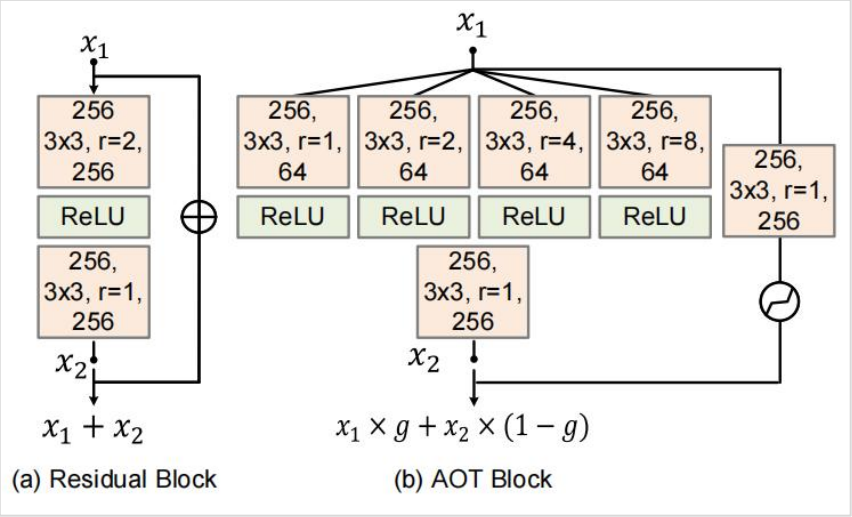


图 2-1 AOT 块的构建

AOT 块采用分裂-转换-合并三步策略。

(1)分裂:如图 2-1(b)所示，一个 AOT 块将一个标准卷积的核分裂成多个子核，每个子核的输出通道较少。例如，将 256 个输出通道的核分裂为 4 个子核，使每个子核具有 64 个输出通道。

(2)转换:每个子核通过使用不同的膨胀率对输入特征 x1 进行不同的变换。具体来说，通过在连续位置之间引入零来展开膨胀的核，这在语义分割领域取得了巨大的成功。使用较大的膨胀率使子核能够“看到”更大的输入图像区域，而使用较小膨胀率的子核则专注于较小感受野的局部模式。

(3)聚合:来自不同感受野的上下文变换最终通过一个连接进行聚合，然后是一个标准的卷积进行特征融合。这样的设计允许 AOT 块通过不同的视图预测每个输出像素。通过以上三个步骤，AOT 块能够聚合多个上下文转换以增强上下文推理。

此外，AOT 块的堆栈在很大程度上丰富了生成器网络中不同路径的组合，允许生成器捕获尽可能多的上下文推理感兴趣的模式。值得注意的是，与最先进的深度学习图像修复模型中使用的标准残差块(图 2-1(a))相比，AOT 块在这三步中没有引入额外的模型参数和计算成本。

受 ResNet[11]巨大成功的启发，深度修复型通常会在其构建块中加入相同的残差连接，以简化网络的训练。如图 2-1(a)所示，相同的残差连接以空间不变的方式通过元素求和来聚合输入特征 x_1 和学习到的残差特征 x_2。然而，它们忽略了缺失区域内外输入像素值的差异，导致了修复图像的颜色差异问题[13]。为了缓解这个问题，在构建块中使用一种新颖的门控残差连接。如图 2-1(b)所示，门控残差连接首先通过标准卷积和 s 型运算从 x_1 计算出空间变化的门值 g，然后 AOT 块将输入特征 x_1 和学习到的残差特征 x_2 与 g 加权求和，表示为

$$X_3 = X_1 \times g + X_2 \times (1-g) \tag{1}$$

与残差块的关系[11]。为了扩大感受野，最先进的修复模型还通过使用空洞卷积作为其构建块来改进残差块(图 2-1(a))。具体来说，他们对一个构建块内的内核使用相同的膨胀率。虽然这种改进的残差块的堆栈可以利用远处的图像上下文，但它经常受到预定义网格模式的编码特征的影响，而不是用于上下文推理的感兴趣的模式[14]。相比之下，本文所提出的 AOT 块能够通过学习具有不同扩展率的聚合上下文转换来捕获丰富的感兴趣模式。

其中，关键构建原理及代码见表 2-1。

表 2-1 AOT 块的构建

项目		内容
分裂转换聚合	原理	将一个标准卷积的核分裂成多个子核，每个子核的输出通道较少，每个子核通过使用不同的膨胀率对输入特征 x1 进行不同的变换。来自不同感受野的上下文变换最终通过一个连接进行聚合，然后是一个标准的卷积进行特征融合。
残差门控	原理	相同的残差连接以空间不变的方式通过元素求和来聚合输入特征 x_1 和学习到的残差特征 x_2。门控残差连接首先通过标准卷积和 s 型运算从 x_1 计算出空间变化的门值 g，然后 AOT 块将输入特征 x_1 和学习到的残差特征 x_2 与 g 加权求和。

代码		`class AOTBlock(nn.Module):`
		`  def __init__(self, dim, rates):`
		`  super(AOTBlock, self).__init__()`
		`  self.rates = rates  #膨胀卷积率[1,2,4,8]`
		`  for i, rate in enumerate(rates):`
		`  self.__setattr__(`
		`  "block{}".format(str(i).zfill(2)),  #填充 0，01，02，04，08`

```
                nn.Sequential(
                    nn.ReflectionPad2d(rate),   #对输入进行反射填充
                    nn.Conv2d(dim, dim // len(rates), 3, padding=0, dilation=rate), #变为(原*1/4,原,原)，包括分裂，转换（膨胀）
                    nn.ReLU(True)
                ),
            )
        self.fuse = nn.Sequential(nn.ReflectionPad2d(1), nn.Conv2d(dim, dim, 3, padding=0, dilation=1))
        self.gate = nn.Sequential(nn.ReflectionPad2d(1), nn.Conv2d(dim, dim, 3, padding=0, dilation=1))
    def forward(self, x):
        out = [self.__getattr__(f"block{str(i).zfill(2)}")(x) for i in range(len(self.rates))]
        out = torch.cat(out, 1)  #将四个子核的输出在第一维度（通道数）上拼接
        out = self.fuse(out)   #对拼接后的特征图进行融合操作
        mask = my_layer_norm(self.gate(x))  #计算门值
        mask = torch.sigmoid(mask)
        return x * (1 - mask) + out * mask #输入特征 x1 和学习到的残差特征 x2 与 g 加权求和
```

2.2 整体网络结构

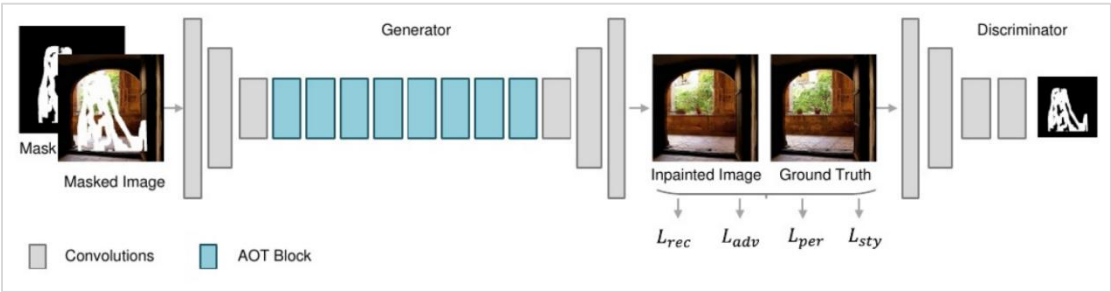


图 2-2 AOT-GAN 网络结构

AOT-GAN 是建立在生成对抗网络(GAN)的基础上的，如图 2-2 所示，该网络由一个生成器和一个鉴别器组成。具体来说，生成器通过堆叠精心设计的块(即 AOT 块)的多层网络来高度模块化，以增强上下文推理。鉴别器由量身定制的掩码预测任务训练，该任务旨在预测下采样的块级修复掩码。模型由重建损失、对抗性损失[12]、感知损失[15]和风格损失[16]联合优化。通过这种联合优化，AOT-GAN 能够对高分辨率图像缺失区域合成合理的内容和清晰的纹理。本文设置 AOT 块数为 6，整体的网络层数为 116 层，在(3,512,512)的图像任务下参数大小约为 3305MB，如图 2-3 所示。

```
            Conv2d-97         [-1, 64, 128, 128]         147,520
              ReLU-98         [-1, 64, 128, 128]               0
  ReflectionPad2d-99        [-1, 256, 136, 136]               0
           Conv2d-100         [-1, 64, 128, 128]         147,520
             ReLU-101         [-1, 64, 128, 128]               0
 ReflectionPad2d-102        [-1, 256, 144, 144]               0
           Conv2d-103         [-1, 64, 128, 128]         147,520
             ReLU-104         [-1, 64, 128, 128]               0
 ReflectionPad2d-105        [-1, 256, 130, 130]               0
           Conv2d-106        [-1, 256, 128, 128]         590,080
 ReflectionPad2d-107        [-1, 256, 130, 130]               0
           Conv2d-108        [-1, 256, 128, 128]         590,080
         AOTBlock-109        [-1, 256, 128, 128]               0
           Conv2d-110        [-1, 128, 256, 256]         295,040
           UpConv-111        [-1, 128, 256, 256]               0
             ReLU-112        [-1, 128, 256, 256]               0
           Conv2d-113         [-1, 64, 512, 512]          73,792
           UpConv-114         [-1, 64, 512, 512]               0
             ReLU-115         [-1, 64, 512, 512]               0
           Conv2d-116          [-1, 3, 512, 512]           1,731
================================================================
Total params: 11,660,355
Trainable params: 11,660,355
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 4.00
Forward/backward pass size (MB): 3256.27
Params size (MB): 44.48
Estimated Total Size (MB): 3304.75
----------------------------------------------------------------
```

图 2-3 网络参数大小

2.3 生成器网络结构

生成器总体分为三部分：编码器、中间层、解码器。在编码器中，对输入数据进行反射填充，提取更多边缘特征，通过三层卷积叠加将特征的通道数放大，同时特征大小缩小 4 倍。相应地，在解码器中使用三层转置卷积进行上采样，将特征图放缩为原通道数和大小以生成修复后图像。在中间层中，采用 6 块 AOT 块(根据 GPU 内存设置)作为生成器的主干。其中，关键原理及代码见表 2-2。

表 2-2 生成器网络的构建

项目		内容
编码器	原理	将输入图像进行反射填充，提取边缘特征，通过三层卷积提取图像细节，将特征的通道数放大，同时特征大小缩小 4 倍。
中间层	原理	堆叠上文构建的 AOT 块以捕获信息丰富的远程上下文和丰富的感兴趣模式，以增强上下文推理。
解码器	原理	使用三层转置卷积进行上采样，将特征图放缩为原通道数和大小以生成修复后图像。

代码

```
class InpaintGenerator(nn.Module):
    def __init__(self):
        super(InpaintGenerator, self).__init__()

        self.encoder = nn.Sequential(
            nn.ReflectionPad2d(3),  #变为(4,518,518)
            nn.Conv2d(4, 64, 7),    #变为(64,512,512)
            nn.ReLU(True),
            nn.Conv2d(64, 128, 4, stride=2, padding=1), #变为(128,256,256)
            nn.ReLU(True),
            nn.Conv2d(128, 256, 4, stride=2, padding=1), #变为(256,128,128)
            nn.ReLU(True),
        )

        self.middle = nn.Sequential(*[AOTBlock(256, rates) for _ in range(block_num
)])
        self.decoder = nn.Sequential(
            UpConv(256, 128), nn.ReLU(True), UpConv(128, 64), nn.ReLU(True), nn.Con
v2d(64, 3, 3, stride=1, padding=1)
        )
        init_weights(self.encoder)
        init_weights(self.middle)
        init_weights(self.decoder)

    def forward(self, x):   #x 是(4,512,512) x = torch.cat([x, mask], dim=1)
        x = self.encoder(x)
        x = self.middle(x)
        x = self.decoder(x)
        x = torch.tanh(x)
        return x
```

2.4 判别器网络结构

在 GAN 模型中，判别器的结构通常比较简单，目的是为了鉴别生成器所生成的输出，指引生成器的优化，因此继承了 PatchGAN[17]中鉴别器的网络架构，因为它在图像翻译领域取得了巨大成功。如图 2-2 所示，判别器网络由几层标准卷积组成，每层卷积将特征图的空间大小减少两倍。鉴别器将来自修复结果或真实数据的图像作为输入，并输出一个预测图。特

别是，预测图的每个像素表示对输入图像中一个 N×N patch 的预测是真还是假。关键代码见表 2-3。

表 2-3 判别器网络结构

<table>
<tr><th colspan="2">项目</th><th>内容</th></tr>
<tr><td>鉴别器</td><td>原理</td><td>鉴别器网络由几层标准卷积组成，每层卷积将特征图的空间大小减少两倍。鉴别器将来自修复结果或真实数据的图像作为输入，并输出一个预测图。</td></tr>
<tr><td>代码</td><td colspan="2">

```python
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.conv = nn.Sequential(
            nn.Conv2d( 3, 64, 4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(64, 128, 4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(128, 256, 4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(256, 512, 4, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(512, 1, 4, stride=1, padding=1),
            nn.Sigmoid()
        )
        init_weights(self.conv)
    def forward(self, x):
        feat = self.conv(x)
        return feat
```

</td></tr>
</table>

2.5 软掩码引导 PatchGAN(SM-PatchGAN)

在实践中，将真实图像表示为 X，对应的二进制修复掩模表示为 m(已知像素值为 0，缺失区域为 1)，⊙为像素相乘，G 为生成器，因此修复结果可以表示为:

$$Z = X \odot (1-\mathrm{m}) + \mathrm{G}(\mathrm{X} \odot (1-\mathrm{m}), \mathrm{m}) \odot \mathrm{m} \tag{2}$$

为了便于细粒度纹理的合成，设计了一个用于鉴别器训练的掩码预测任务，我们称之为

软掩码引导 PatchGAN(SM-PatchGAN)。具体来说，我们对修复掩码进行下采样，作为掩码预测任务的真实值。来模拟像素在缺失区域的边界周围传播，如图 3-4 所示，使用软 patch 级掩码进行训练。通过高斯滤波得到软掩模。表示判别器的对抗性损失为:

$$L_{adv}^{D} = \mathrm{E}_{z\sim p_z}[(D(z)-\sigma(1-m))^2] + \mathrm{E}_{x\sim p_{data}}[(D(x)-1)^2] \tag{3}$$

式中 σ 为下采样与高斯滤波的复合函数。对应地，生成器的对抗损失表示为:

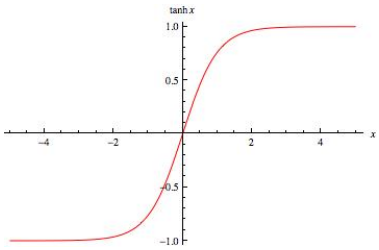
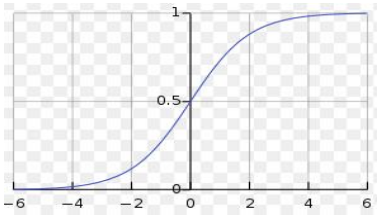
$$L_{\mathrm{adv}}^{G} = \mathrm{E}_{z\sim p_z}[(D(z)-1)^2 \odot \mathrm{m}] \tag{4}$$

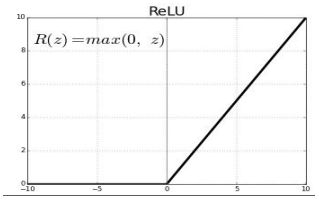
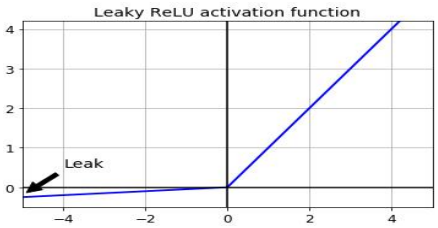
其中，仅使用对缺失区域的合成补丁的预测来优化生成器。通过这样的优化，鼓励鉴别器将合成的缺失区域的补丁从缺失区域之外的真实上下文的补丁中分割出来。增强的鉴别器反过来可以促进生成器合成更真实的纹理。

2.6 相关激活函数

表 2-4 是这次 DCGAN 网络模型构建所需要的相关激活函数。

表 2-4 AOT-GAN 的相关激活函数

激活函数	函数图像	应用
Tanh		经过 tanh 函数后，输出范围为-1 到 1.适用于输出需要为-1 到 1 的层
Sigmoid		sigmoid 函数当输出为 0 和 1 时曲线趋向平缓，输出结果将大部分为 0 或者 1.适合二分类问题

Relu		Relu 函数比 sigmoid 收敛的快，但是由图可知，输出值为正数
LeakyReLU		修正版的 Relu 函数，添加了很小的负区域增量。可以适用输入含有负数的问题

三、损失函数

选择图像修复损失函数的原则是保证每个像素重建的精度和修复图像的视觉保真度，为此，仔细选择了四个优化目标，即 L1 损失、风格损失[16]、感知损失[15]和 SM-PatchGAN 的对抗损失，用于 AOT-GAN，遵循大多数现有的深度学习图像修复模型。首先，我们包括一个 L1 损失，以确保像素级的重建精度:

$$L_{\mathrm{rec}} = \left\| x - G(x \odot (1\text{-}\mathrm{m}), \mathrm{m} \right\|_1 \tag{5}$$

由于感知损失[15]和风格损失[16]用于图像修复的有效性已被广泛验证，本文也将它们纳入以提高感知重建的精度。具体来说，感知损失旨在最小化被修复图像和真实图像的激活图之间的 L1 距离:

$$L_{per} = \sum_i \frac{\left\| \phi_i(x) - \phi_i(z) \right\|_1}{N_{\mathrm{i}}} \tag{6}$$

其中 Φ_i 是来自预训练网络(例如 VGG19)的第 i 层的激活图 Ni 是 Φ_i 中的元素数量。类似地，风格损失被定义为修复图像和真实图像深度特征的 Gram 矩阵之间的 L1 距离:

$$L_{\mathrm{sty}} = \mathrm{E}_i[\left\| \phi_i(x)^T \phi_i(x) - \phi_i(z)^T \phi_i(z) \right\|_1] \tag{7}$$

最后，加入了章节 3.5 中描述的 SM-PatchGAN 的对抗性损失以提高手绘图像的视觉保真度。整个 AOT-GAN 是通过这四个目标的联合优化来训练的，我们将整体优化目标总结为:

$$L = \lambda_{\mathrm{adv}} L_{adv}^G + \lambda_{rec} L_{rec} + \lambda_{per} L_{per} + \lambda_{sty} L_{sty} \tag{8}$$

根据实验结果，选择参数 λ_{adv}=0.01，λ_{rec}=1，λ_{per}=0.1，λ_{sty}=250 进行训练。

四、数据集及其预处理

4.1 人脸数据集介绍

CelebAMask-HQ 是 CelebFaces Attribute 的缩写，由香港中文大学开放提供，广泛用于人脸相关的计算机视觉训练任务。CelebA-HQ 数据集中提取并标注的高质量人脸图像数据集，主要用于面部属性分析、面部分割、图像生成和编辑等任务。它包含了对面部图像进行精细分割标注的掩码（mask），覆盖了面部的不同区域和属性。

数据集特点:

(1)高分辨率图像：CelebAMask-HQ 的数据集图像分辨率为 1024×1024 像素，提供了比原始 CelebA 数据集更高质量的图像。

(2)精细的分割掩码：每张图像都附带有精细标注的分割掩码，这些掩码覆盖了脸部的不同部分，如皮肤、头发、眼睛、眉毛、鼻子、嘴巴、耳朵、眼镜等。

(3)多种面部属性：数据集中包含了面部的多种属性标注，例如性别、年龄、发型、表情等，使其在多种任务中具有很高的实用性。

(4)丰富的样本量：CelebAMask-HQ 包含了 30,000 张高质量人脸图像，这些图像是从 CelebA-HQ 数据集中挑选并标注的。

4.2 数据预处理

数据总体分为人脸图像 face 和对应掩码图 face-mask,其中，face 路径下文件如图 4-1 所示，包含 2000 张人脸图片，其分辨率为 1024*1024，命名格式为数字字符串 0-1999，

图 4-1 face 数据集

Face-mask 路径下文件如图 4-2 所示，包含 24796 张单通道掩码图，在 face 路径下的每张人脸图都对应数十张 mask 掩码图，掩码图的命名格式为“五位编号_遮挡区域”，其中五位编号代表对应的 face 图像编号，在这里范围是 00000-01999，遮挡区域代表 mask 掩码部分遮挡人像的区域，包含了

“l_brow,neck,u_lip,l_lip,eye_g,mouth,hat,ear_r,hair,r_eye,neck_l,r_brow,skin,l_eye,l_ear,r_ear,cloth,nose”

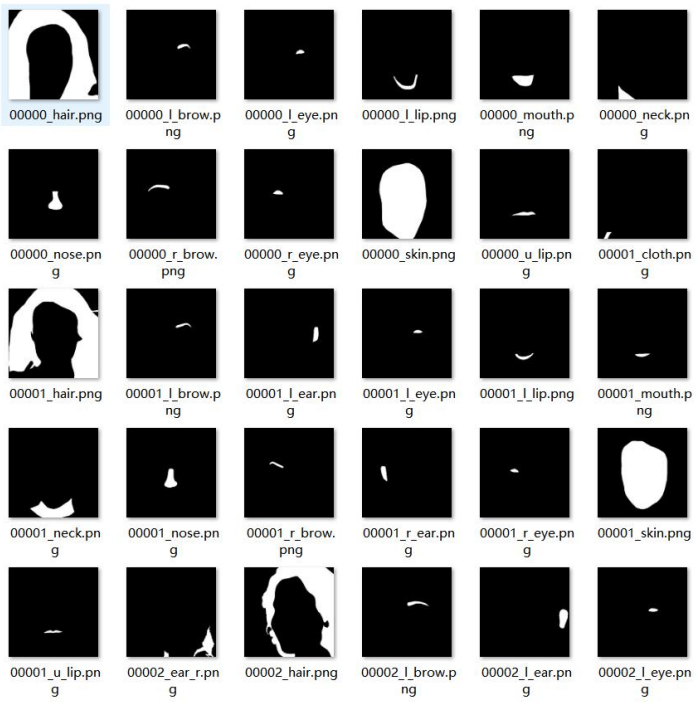


图 4-2 face-mask 数据集

4.2.1 数据选择

在训练时，将在每轮训练前随机选择 face 对应的 mask 的其中一张，譬如在第一轮训练时“0.jpg”选择 00000_nose.png 作为掩码图，在第二轮训练时对于“0.jpg”选择“00000_mouth.png”作为掩码图，以此类推。

4.2.2 数据操作

由于硬件设备有限，在训练中无法承担 1024*1024 像素的图像任务，因此将其重塑大小为 512*512 的图像，采用最近邻插值算法进行调整。随后进行对 face 图像和 mask 掩码图进行随机水平反转，增加数据多样性，并对其进行随机旋转，旋转角度范围为(0,45),转换为 tensor 张量，将 face 的 tensor 张量映射到[-1，1]。随后在训练前将掩码图与人像图进行融合，将人脸对应掩码的缺失部分置 1，生成带掩码的人脸图，其原理为:

$$\text{Masked_img} = (\text{face}*(1-\text{mask}))+\text{mask} \tag{9}$$

最后，将生成的带掩码的人脸图与 mask 掩码图在通道上进行拼接，变成 4 通道的输入特征，为了符合 pytorh 深度学习框架的数据格式，在输入特征维度上进行增加，使得输入最终变为(batch_size,4,512,512)的 tensor 张量。

五、实现细节

5.1 实验环境配置

5.1.1 硬件环境

CPU：AMD Ryzen 5800H

显卡：NVIDIA GeForce RTX 3060 6GB

内存：16GB

5.1.2 软件环境

操作系统：Windows10

开发语言：Python3.8

编程软件：VSCode v1.85.0

深度学习框架：Pytorch-gpu

5.2 训练过程

在模型训练中，设置批次大小为 1(gpu 内存受限)，每次随机采样掩码。对于生成器和判别器,前 100 个周期学习率设置为 2e-4,第 101 至 150 周期设置学习率为 5e-5,第 151-200 周期设置学习率为 1e-5。使用 β_1=0 和 β_2=0.9 的 Adam 优化器进行训练。在计算损失时，使用 ImageNet 上预训练的 VGG19 作为预训练的网络，用于计算风格损失 L_{sty} 和感知损失 L_{per}。网络的初始化采用 kaiming 初始化。在训练过程中，每个周期保存一次检查点便于观察模型性能的成长，并导出生成器和判别器的损失。

六、实验结果分析

6.1 损失函数结果

如图 6-1，生成器的 L1 损失、感知损失、风格损失总体下降并趋于收敛，其中，L1 损失收敛于 0.023，风格损失收敛于 0.032，感知损失收敛于 0.055，生成器与判别器的对抗损失收敛于某中间值。由图像可知，生成器和判别器网络的损失在一定的范围内变换分布比较均匀，证明了其在进行有效对抗，生成结果在不断优化。双方趋势平稳。说明网络的生成能力和判断精度都在不断提高。

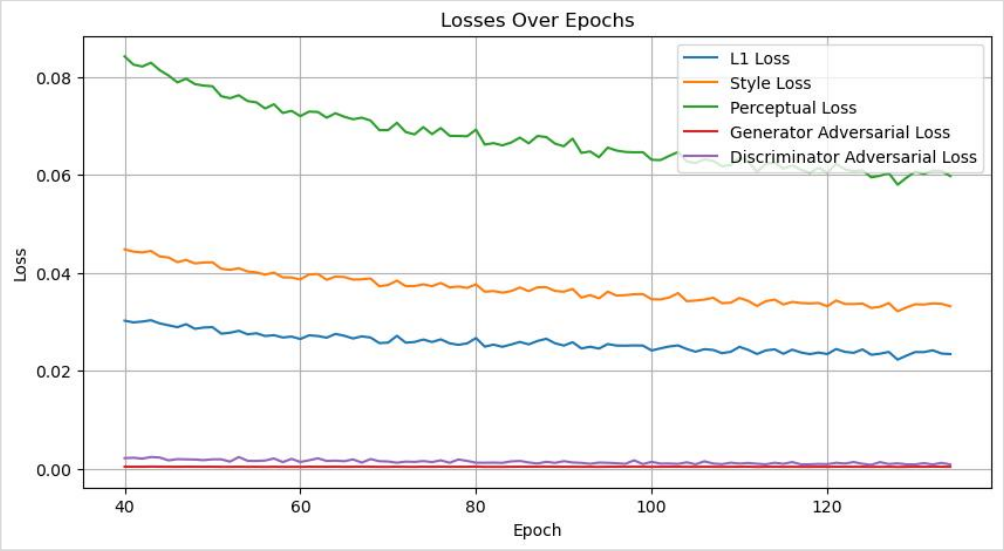


图 6-1 模型损失函数

6.2 直观修复感受

使用生成模型修复 700 张图片，以人眼感官判断修复效果见表 6-1。

表 6-1 修复结果表

判断标准	数字/比例
修复图片看起来像人的	687/98%
能找出来修复痕迹的	441/62%
看不出来是修复图像还是原图的	511/73%
能明显看出来是修复图像的	84/12%

6.3 部分修复结果展示

图 6-2 至 6-6 展示了一系列典型的、实例的图片修复效果。其中从左到右分别代表 face 人脸图、带掩码的人脸图、修复图、mask 掩码图。

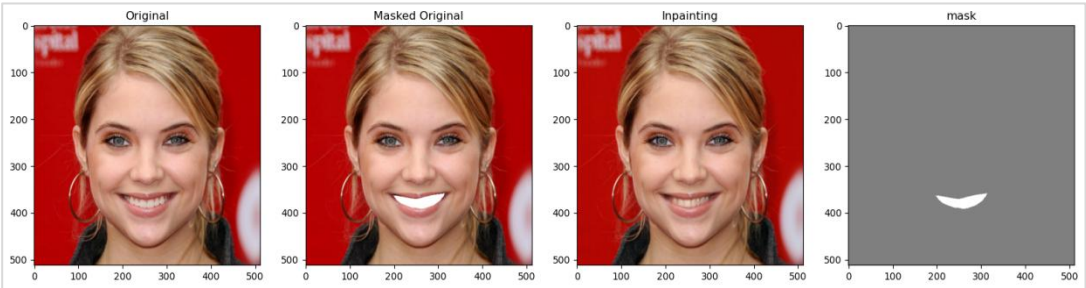


图 6-2 model_135_teeth 修复图

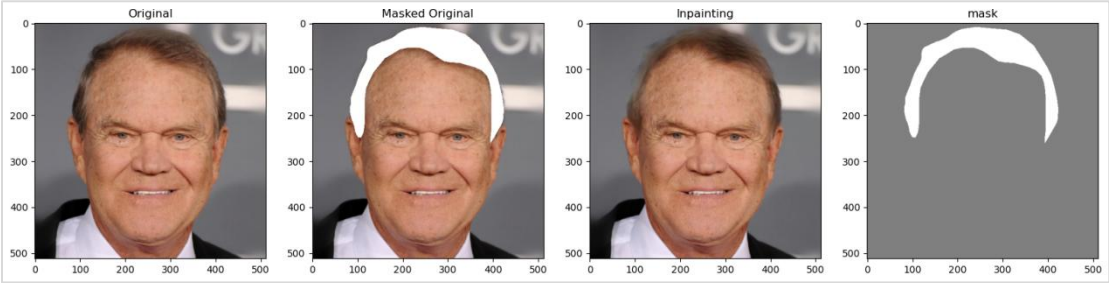


图 7-3 model_114_hair 修复图

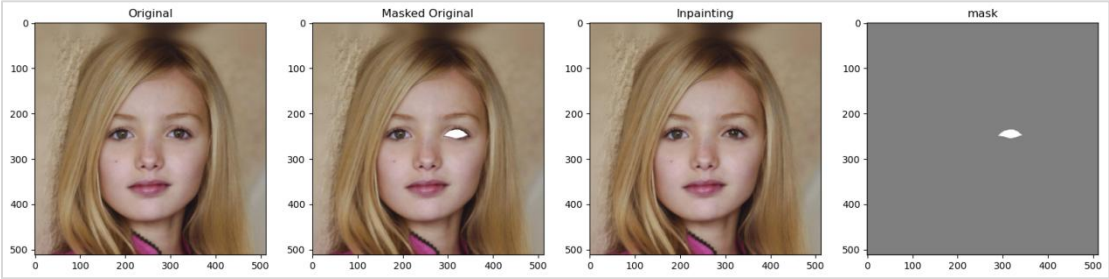


图 7-4 model_114_eye 修复图

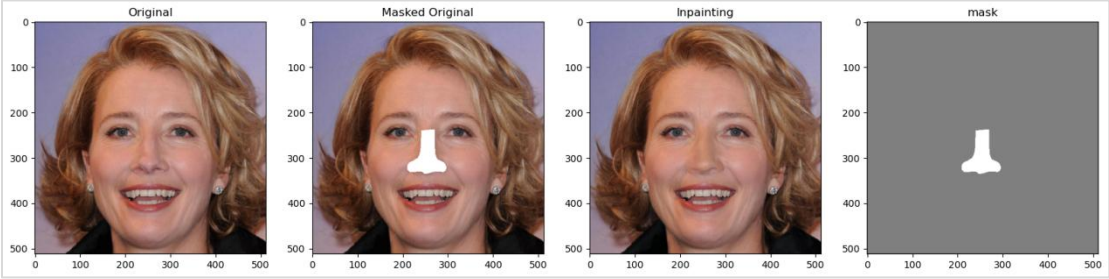


图 7-5 model_135_nose 修复图

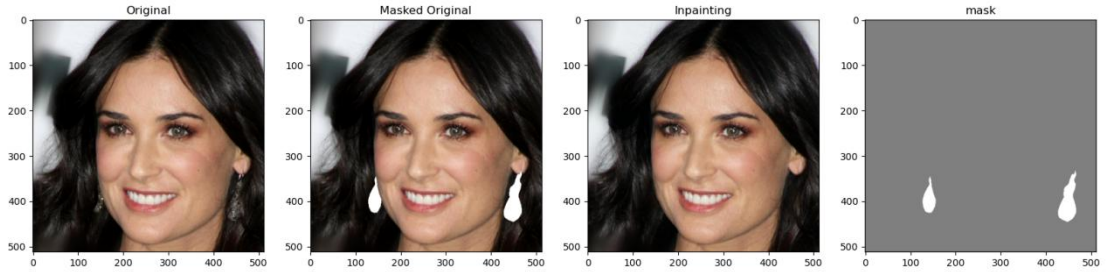


图 7-6 model_135_earring 修复图

6.4 修复案例分析

经过 200+小时的模型训练之后，网络的修复效果已经尤为可观，对于大部分清晰的正脸图片，直接生成的假图片已经到了不仔细看就意识不到差别的地步。当然，对于大面积掩码的图片，还是难以做到高清保真效果，譬如大面积的头发缺失，模型就无法准确根据上下文语义推测出头发的纹理细节，因此导致局部模糊纹理。

部分修复结果也会出现与原图不同风格的情况，譬如眼睛的瞳孔颜色不同，这可能是由于受整体训练集的眼睛瞳孔颜色的影响，使模型学习到统一的颜色而无法具有特殊性。

以上问题，大多是因为训练集趋向不够明显，数量不够大。这次实验对数据集的构建仍有提高的余地，如果条件允许，增加模型的深度也可以提高效果。

七、总结与展望

本文从选题背景出发介绍了相关情况和选题动机。对一些常见的深度学习网络的原理进行了总结，写出了自己的体会。介绍了从 CNN 网络和 GAN 网络中发展出来的 AOT-GAN 网络，结合本文的主体从头开始复现基于 AOT-GAN 图像修复的原理，并一步一步地实现了这些原理。最后按照搭建出来的 AOT-GAN 网络进行了训练，实验，和优化调整。经过不断地微调与改进之后，实现了较为理想的修复效果。

未来的工作可以从以下几个方面展开：

(1)数据集扩展：当前实验中使用的数据集在数量和多样性上仍有提高的空间。增加更多种类的训练数据可以提升模型的泛化能力，从而在不同场景下表现更佳。

(2)模型优化：进一步优化 AOT-GAN 的架构，特别是针对大面积掩码区域的修复效果，可能需要引入更高级的上下文理解和细节推理机制。

(3)计算资源：提升硬件配置，允许处理更高分辨率的图像，进一步提高修复效果。当前由于硬件限制，图像尺寸被调整为 512x512，未来可以尝试更高分辨率的处理。

(4)应用拓展：将 AOT-GAN 应用于其他类型的图像修复任务，如医学影像、历史照片修复等，验证其在不同领域的通用性和有效性。

参考文献

[1] 卢宏涛,张秦川.深度卷积神经网络在计算机视觉中的应用研究综述[J].数据采集与处理,2016,31(01):1-17.

[2] 李炬. 基于生成对抗网络的图像修复技术研究[D].西南交通大学,2019.

[3] Iizuka S, Simo-Serra E, Ishikawa H. Globally and locally consistent image completion[J]. ACM Transactions on Graphics, 2017, 36(4): 107.

[4] Li Y, Liu S, Yang J, et al. Generative face completion[C]// Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Hawaii, July 21-26, 2017. Piscataway: IEEE, 2017: 3911-3919.

[5] 曹仰杰,贾丽丽,陈永霞,林楠,李学相.生成式对抗网络及其计算机视觉应用研究综述[J].中国图象图形学报,2018,23(10):1433-1449.

[6] Pathak D, Krhenbühl P, Donahue J, et al.Context encoders:feature learning by inpainting[C]//Proceedings of 2016 IEEEConference on Computer Vision and Pattern Recognition.Las Vegas, NV, USA:IEEE, 2016:2536-2544.[DOI:10.1109/CVPR.2016.278]

[7] 王永飞.一种图像缺损修复算法分析[J/OL].激光与光电子学进展:1-19[2020-04-05].http://kns.cnki.net/kcms/detail/31.1690.TN.20191106.1205.060.html

[8] 尹悦. 数字图像修复技术在大麦地岩画修复中的应用[D].宁夏大学,2019.

[9] 范春奇,任坤,孟丽莎,黄泷.基于深度学习的数字图像修复算法最新进展[J].信号处理,2020,36(01):102-109.

[10] 马悦. 基于深度生成模型的人脸图像修复算法研究[D].西安电子科技大学,2019.

[11] K.He, X.Zhang, S.Ren, and J.Sun,“Deep residual learning for image recognition,”in CVPR, 2016, pp.770-778.1,3,4,,5.10

[12] I.Goodfellow, J.Pouget-Abadie, M.Mirza, B.Xu, D.Warde-Farley,S.Ozair, A.Courville, and Y.Bengio,“Generative adversarial nets,"in NeurIPS, 2014, pp.2672-2680.1,3,5

[13] J.Yu,Z.Lin,J.Yang,X.Shen,X.Lu,and T.S.Huang.“Free-form image inpainting with gated convolution,” in ICCV, 2019, pp.4471-4480.1,2,3,4,6,7,8,9,10

[14] J.Yu,Z.Lin,J.Yang,X.Shen,X.Lu,and T.S.Huang.“Generative image inpainting with contextual attention,”in CVPR, 2018,pp.5505-5514.1,2,3,5,6,7,8

[15] J.Johnson,A.Alahi,and L.Fei-Fei,“Perceptual losses for real-time style tra-nsfer and super-resolution,in ECCV 2016, pp.694-711.3,6

[16] L.A.Gatys,A.S.Ecker,and M.Bethge,“Image style transfer using convolutional neural networks,”in CVPR,2016,pp.2414-2423.3,6

[17] P.Isola,J.-Y.Zhu,T.Zhou,and A.A.Efros,“Image-to-image translation with co-nditional adversarial networks,”in CVPR, 2017, pp.1125-1134.2,3,5,6,7,10